

Port Wrangler — Code-Grounded Technical Roadmap

Produced by Claude (Opus 4.8) running in Claude Code, with full read access to the actual com.dbdeployer source tree at commit cafd191. Where this document conflicts with the four prior AI research outputs (docs/cgpt-deep-research-report.md, docs/deepseek-research.md, docs/claude_ai_management_tool.md, and backend/docs/port-wrangler-roadmap.md), **this one is correct about the codebase** — the others were written without seeing the source and contain grounding errors flagged below.

How this document is different

The four existing roadmaps are strong on *aspiration* but were generated against the public GitHub README, not the code. That produces three classes of error: (1) recommending things you already did, (2) recommending the wrong tool for your actual stack, and (3) violating your own stated constraints. Every class reference, dependency, and schema decision below was verified against the source. Spring AI is pinned to **1.0.3** (the latest 1.0.x GA patch as of October 2025) to satisfy your "GA only" constraint exactly.

Post-research scope changes incorporated:

- **Kafka management module dropped** — extracted into a separate, dedicated Kafka tool project. Port Wrangler still *deploys* Kafka; it no longer *manages* it. The roadmap saves ~3 weeks and gains a sharper thesis.
- **Five new features added** (from conversation + Odysseus research): Model Cookbook, Model Comparison, MCP server exposure, Docker Compose export, and "What's my stack" agent tool.

0. Corrections to the existing roadmaps (read this first)

These are not nitpicks — each one changes a Phase-0/Phase-3 decision.

#	Claim in prior docs	Reality in the code	Correction
1	"Replace ddl-auto with Flyway " (consolidated roadmap §0.3, §4)	You use Liquibase . liquibase-core is on the classpath (build.gradle.kts:37 ../backend/build.gradle.kts#L37); changesets V1–V4 exist under db/changelog/changes/ ../backend/src/main/resources/db/changelog/changes/ . But the Liquibase config is <i>commented out</i> (application.yml:39-40 ../backend/src/main/resources/application.yml#L39-L40) and ddl-auto: update is active (application.yml:31 ../backend/src/main/resources/application.yml#L31). Hibernate is managing	Do not use Flyway. the Liquibase cutover started: baseline the current schema.

	the schema right now, and the entities have already drifted from V1__init.sql (the live DeploymentPipeline has deployment_container_id and is_template; V1 has neither).	spring.li flip ddl-e validate. §7.
springAiVersion = "1.0.0-M5"		Pin 1.0.0
2 (deepseek-research.md:58 (deepseek-research.md#L58))	M5 is a milestone . This directly violates your hard constraint "Spring AI 1.0 GA only."	1.0.x GA §2 and t Appendi
3 "14 database types" (your own framing, echoed everywhere)	DbType (../backend/src/main/java/com/dbdeployer/model/DbType.java) has 26 entries: databases plus Kafka, RabbitMQ, Conduktor, Grafana, Prometheus, Loki, MinIO, Keycloak, Vault, Nginx, Adminer, pgAdmin.	Repositi whole pr Port Wr already general infra de not a DE This ma strength New lon running (model p should r the Dep SPI , not a paralle mechan See §6 I 2.
"Build a 4-step async deploy 4 pipeline" (presented as future work)	Already built and battle-tested: ImagePull → ContainerCreate → ContainerStart → Finalise, DB-persisted, with afterCommit firing (PipelineOrchestrator (../backend/src/main/java/com/dbdeployer/pipeline/PipelineOrchestrator.java)), @Async execution + post-commit re-fetch (PipelineRunner (../backend/src/main/java/com/dbdeployer/pipeline/PipelineRunner.java)), and a clean DeployStep SPI.	the Dep SPI , not a paralle mechan See §6 I 2.
MCP starter spring-ai-mcp- 5 client-spring-boot- starter (old naming)	GA renamed all starters to the spring-ai-starter-* prefix.	Use spri starter-n client. S
"No special flags needed for Apple 6 Silicon Metal" stated as fact for <i>containers</i>	True for native Ollama on macOS; false for Dockerised Ollama — Docker Desktop on macOS runs a Linux VM with no GPU passthrough , so a containerised Ollama on a Mac runs CPU-only .	Detect tl warn the recomm native O on Mac, containe on Linux/N See §6 I 2.

1. Competitive differentiation map

Port Wrangler competes in four lanes at once. In any single lane it is *not* the deepest tool — and that is fine, because **no competitor spans all four lanes, and none has an AI agent with first-class tools over the infrastructure it provisions**. That intersection is the moat.

1.1 Capability matrix

Legend: ● full / first-class · ◐ partial · ○ none.

Tool	License	DB deploy + lifecycle	Kafka mgmt	Local-LLM mgmt	Model Cookbook	AI agent over <i>your</i> stack	MCP server (IDE integration)	Native ins.
Port Wrangler	OSS (yours)	● (26 types)	○ (separate project)	○→● (Phase 2)	○→● (Phase 2)	○→● (Phase 5)	○→● (Phase 5)	● jpack DMG/
Portainer	OSS/CE	◐ (generic containers)	○	○	○	○	○	◐ (co
DbGate	OSS	○ (connects, doesn't deploy)	○	○	○	○	○	●
Adminer	OSS	○ (connects)	○	○	○	○	○	◐ (1 f file)
Beekeeper Studio	OSS/comm	○ (connects)	○	○	○	○	○	●
Kafbat UI (ex-kafka-ui)	Apache-2.0	○	●	○	○	○	○	◐ (co
AKHQ	Apache-2.0	○	●	○	○	○	○	◐ (conta
Redpanda Console	BSL (source-avail)	○	●	○	○	○	○	◐
Conduktor	Commercial	○	● (+governance)	○	○	○	○	◐
Ollama	OSS	○	○	● (runtime+models)	○	○	○	●
Open WebUI	OSS	○	○	◐ (chat UI over Ollama)	○	○	○	◐ (co
LM Studio	Free (closed)	○	○	●	◐ (basic)	○	○	●
Jan	OSS	○	○	●	◐ (basic)	○	○	●
Odysseus	OSS	○	○	●	● (Model Cookbook)	◐ (generic chat)	○	◐ (co
Docker Model Runner	OSS (in Docker Desktop)	○	○	● (OCI models)	○	○	○	● (bur

1.2 The gap, stated precisely

A backend developer starting a microservices project today opens, in separate windows: **Portainer or docker run** (infra), **a DB client** (DBeaiver/DbGate), **a Kafka UI** (Kafbat/AKHQ), and **Ollama + a chat UI** (Open WebUI). Four tools, four mental models, zero shared context, and *none* of them can act on your behalf.

Port Wrangler collapses that into one local-first app — and then does the thing none of them do: an **AI assistant whose tools are the infrastructure layer**. "Deploy a Redis, create a Kafka topic orders with 6 partitions, pull llama3.1:8b, and give me the Spring config" becomes one sentence, executed (with confirmation on writes) against the very services the tool manages, with RAG grounding over your own deployment history and container logs.

1.3 Honest differentiation (defensible in an interview)

- **Genuine moat:** the *agentic seam*. Competitors are viewers; Port Wrangler is an operator. The @Tool layer wraps the already-existing [DbInstanceService](#) ([../backend/src/main/java/com/dbdeployer/service/DbInstanceService.java](#)) surface (deploy, start/stop/removeInstance, getLogs, discoverContainers, rename, syncStatuses) — so the agent is thin and the actions are the same battle-tested code paths the UI uses. That is a strong design story.
- **Real, not vibes:** you already ship native installers (jpackage DMG/EXE), a DB-persisted recovery-on-restart pipeline, container discovery/import, and live per-container metrics (CPU/mem/net/blkio/health via getContainerMetrics). Most "show HN" infra tools are a Docker wrapper; yours has depth.
- **Where you should *not* claim parity (say so up front):** you are not a SQL query editor (DbGate wins), not a Kafka governance/RBAC/data-masking platform (Conduktor/Kpow win), and not a multi-node production orchestrator (Portainer/k8s win). Port Wrangler is the **local pre-development cockpit**. Owning that niche cleanly beats being a worse Portainer.

2. Validated technology decisions

Format: **Verdict** → reasoning → what to actually do.

2.1 Vector store: pgvector — CONFIRM (strong)

pgvector wins here for a reason most comparisons miss: it is the only option that satisfies your **zero-extra-service** principle *and* gives you transactional consistency with chat history.

- **vs Chroma:** Chroma is a separate Python service + its own store. That is a second runtime and a second backup/lifecycle surface on the developer's machine. It directly violates zero-extra-service.
- **vs Pinecone/Weaviate:** Pinecone is cloud-only (dead on arrival for a local-first tool). Weaviate is another container to run and sync. Both also mean embeddings live *outside* the transaction that writes the chat message — so a crash between "save message" and "save vector" desynchronises them. With pgvector, the message row and its embedding commit together.
- **You already provision PostgreSQL 16.** [SystemDbProvisioner](#) ([../backend/src/main/java/com/dbdeployer/config/SystemDbProvisioner.java](#)) launches postgres:16 on host port 5499 before Spring boots. pgvector is one CREATE EXTENSION in a Liquibase changeset — no new container, no new port, no new dependency surface.
- **The one real limitation, and why it doesn't bite you:** pgvector's ANN index quality degrades relative to dedicated stores beyond ~1–10M vectors. At local-dev scale (deployment records + a few

thousand log chunks + chat history = low tens of thousands of vectors) this is irrelevant. HNSW on pgvector will return in single-digit milliseconds.

- **Do:** spring-ai-starter-vector-store-pgvector, HNSW index, COSINE_DISTANCE, dims from the EmbeddingModel. (Details in §4/§7.)

2.2 LLM runtime: Ollama default — CONFIRM, but build a runtime abstraction

Ollama is the right *default*, but because Port Wrangler is Docker-native, **Docker Model Runner (DMR) deserves to be a first-class second runtime**, not an afterthought.

- **Why Ollama default:** OpenAI-compatible API, the best Spring AI starter (spring-ai-starter-model-ollama gives you both chat *and* embeddings), ~52M monthly downloads, the broadest model library, and universal ecosystem support. It is the lowest-friction path to a working chat in Phase 3.
- **Why DMR is not optional fluff:** it shipped in Docker Desktop 4.40 (mid-2025), stores models as **OCI artifacts** on Docker Hub, and is Compose-native. Your entire product is "treat infra as managed Docker containers" — DMR fits that worldview *better* than Ollama does. Modelling it as a peer runtime is a 1-day abstraction that doubles your "look how extensible this is" story.
- **vs LocalAI:** capable and OpenAI-compatible, but a smaller ecosystem and no Spring AI first-party starter. Relegate to "community runtime," not a launch target.
- **Do:** introduce a ModelRuntime enum (OLLAMA, DOCKER_MODEL_RUNNER) that extends the same catalog pattern as DbType/DatabaseCatalog, and a ModelRouter that produces a ChatClient bound to the chosen runtime + model. The abstraction costs little and reads as senior design. (§4.)

2.3 Embedding model: nomic-embed-text — CONFIRM as default, make it configurable

- **Why nomic:** 768 dims, Apache-2.0, strong quality-per-millisecond on local CPU, runs in the same Ollama you already deploy (one runtime, two roles). 768 dims is comfortably under pgvector's HNSW 2000-dim ceiling and keeps the index small.
- **Upgrades worth a config switch, not a default:** mxbai-embed-large (1024d, higher retrieval quality) and embeddinggemma are better on benchmarks but cost more memory/latency on a laptop.
For *infrastructure/technical* text (deployment configs, log lines, topic metadata) the marginal recall gain does not justify the default footprint.
- **The gotcha to design around:** a pgvector column's dimension is **fixed at table creation**. If a user switches embedding models, the dimension may change and every stored vector must be re-embedded. So: pick the model+dimension at install time, store it in config, and make "switch embedding model" an explicit, destructive re-index operation — never a silent setting. (§7.)

2.4 AI abstraction: Spring AI 1.0 GA — CONFIRM, pin 1.0.3

Spring AI 1.0 GA (released 2025-05-20) gives you exactly the surface this roadmap needs, all first-party: ChatClient, ChatMemory + MessageWindowChatMemory + JdbcChatMemoryRepository, the advisor chain (MessageChatMemoryAdvisor, QuestionAnswerAdvisor, VectorStoreChatMemoryAdvisor), @Tool + ToolCallingManager (for the confirmation flow — §5), PgVectorStore, and an MCP client.

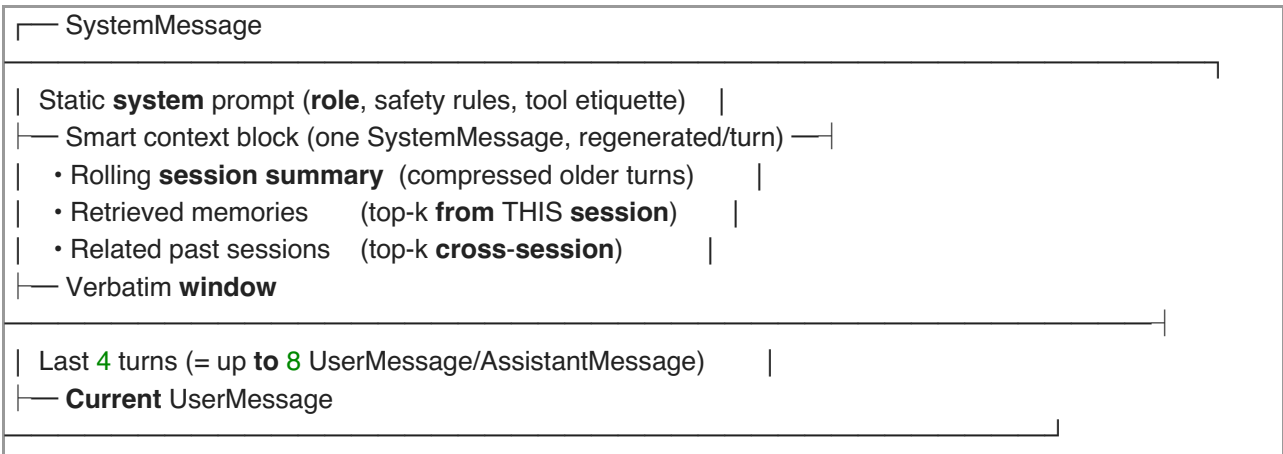
- **Pin 1.0.3** (latest 1.0.x GA patch, Oct 2025). This satisfies "GA only" with no ambiguity.
 - **Note for later:** a 1.1.x line exists (in milestone/RC at the time of this research). Staying on 1.0.3 now is correct per your constraint; the eventual 1.0.3 → 1.1.x bump is a small migration (mostly the advisor builder API), not a rewrite. Don't pre-adopt it.
-

3. Token optimization — Java implementation design

This is the section to go deepest on (your constraint: "prioritise depth in one area"). The goal is a Java/Spring AI implementation of open-relay's layered context assembly so per-message token cost stops growing linearly with conversation length.

3.1 Target prompt shape (the open-relay layering, in Spring AI terms)

Every model call assembles this, top to bottom:



The verbatim window is bounded (constant cost). The summary grows sub-linearly (it's re-compressed, not appended). Retrieval is fixed-k. So total prompt tokens flatten instead of climbing — which is exactly why open-relay reports 30–35% savings at 10 turns and 70–80% at 40+.

3.2 Where it plugs into Spring AI

Implement it as a **custom advisor** plus a **custom ChatMemory**, composed in the advisor chain. Do **not** try to do this inside a controller — advisors are the supported seam and they run for both call() and stream()).

```
ChatClient.prompt()
  .advisors(
    contextAssemblyAdvisor, // order = 100 — builds the smart-context SystemMessage
    infraRagAdvisor)       // order = 200 — QuestionAnswerAdvisor over the infra KB
  // ToolCallingAdvisor is auto-registered LAST by Spring AI
  .stream()
```

In Spring AI 1.0 GA the advisor contract is CallAdvisor / StreamAdvisor; the convenience base class BaseAdvisor exposes before(ChatClientRequest, AdvisorChain) and after(ChatClientResponse, AdvisorChain) so you implement one method for both call and stream. Lower getOrder() runs first; memory/context advisors must sit **outside** (before) the tool-calling advisor.

3.3 Class structure

Class	Stereotype	Responsibility	Key methods
ContextAssemblyAdvisor	@Component BaseAdvisor	Orchestrates the smart-context block; injects it as a SystemMessage ahead of the	ChatClientRequest before(ChatClientRequest,

		verbatim window	AdvisorChain); int getOrder()
RollingWindowChatMemory	@Component implements ChatMemory	The verbatim window (delegates to MessageWindowChatMemory) plus summary-aware get	add(convId, msgs), get(convId), clear(convId)
RollingSummaryService	@Service	Decides when to summarise and calls the LLM to compress	Optional<SessionSummary> summariseIfNeeded(String sessionId)
MemoryRetriever	@Service	pgvector retrieval + open-relay composite re-ranking	List<ScoredChunk> retrieve(RetrievalQuery q)
RecencyFrequencyScorer	@Component	Pure function: composite score from the three signals	double score(double semantic, Instant lastSeen, int accessCount)
TokenBudget	@Component	Token estimation + budget arithmetic for triggers	int estimate(String text), int estimate(List<Message>)
ChatSession, ChatMessage	@Entity	Persistence (see §7 for tables)	—

3.4 The summarization trigger — hybrid (turns and tokens)

A single trigger is wrong: "every N turns" over-summarises short chatty turns and under-summarises long ones; "every X tokens" can let turn count drift. Use **both**, whichever fires first:

```
boolean shouldSummarise(ChatSession s, int verbatimWindowTokens) {
    int newTurnsSinceSummary = s.getCurrentSeq() - s.getSummarizedThroughSeq();
    return newTurnsSinceSummary >= SUMMARY_EVERY_N_TURNS // = 4 (open-relay's cadence)
        || verbatimWindowTokens >= VERBATIM_TOKEN_BUDGET; // = 1500
}
```

Summarisation runs **after** the assistant reply is persisted, off the request hot path (an @Async method, mirroring how PipelineRunner already runs deploy work asynchronously). The user never waits for compression.

3.5 The rolling summarization prompt (send this to a small/fast model)

Use a *cheap* model for compression (e.g. llama3.2:3b or whatever small model is pulled), not the chat model — summarisation is the high-frequency, low-stakes call.

SYSTEM:

You maintain a running summary of a technical conversation between a developer **and** an infrastructure assistant for "**Port Wrangler**" (a local Docker/Kafka/LLM management tool). Rewrite the PRIOR SUMMARY plus the NEW TURNS into a single updated summary.

Rules:

- Preserve every durable fact: **instance** names, db types, ports, Kafka topic names **and** partition counts, model names/tags, connection strings, **and** any decision the user made.
- Preserve unresolved intents ("user still wants to...", "pending: confirm removal of X").
- Drop pleasantries, restated questions, **and** anything already acted on **and** closed.
- Never invent facts. If a value is unknown, omit it.
- Output ≤ 200 tokens, as terse bullet points. No preamble.

USER:

PRIOR SUMMARY:

{priorSummary | "(none)"}

NEW TURNS:

{the N turns being compressed, role-tagged}

The result replaces chat_session.rolling_summary; bump summarized_through_seq and summary_token_count. Because the summary is *rewritten* (not appended), its size is bounded by the ≤ 200 -token instruction — this is what makes cost sub-linear.

3.6 Retrieval scoring — the open-relay formula in pgvector

$\text{score} = 0.35 \cdot \text{semantic} + 0.35 \cdot \text{recency} + 0.30 \cdot \text{frequency}$

pgvector can only sort by vector distance, so the pattern is **over-fetch by vector, re-rank in**

Java: pull the top k-4 candidates by cosine distance, then apply the composite score and take top-k.

```
// MemoryRetriever
List<ScoredChunk> retrieve(RetrievalQuery q) {
    float[] qv = embeddingModel.embed(q.text());
    var candidates = jdbc.query(CANDIDATE_SQL, rs -> map(rs),
        q.sessionId(), toPg(qv), q.metadataType(), q.k() * 4); // over-fetch
    Instant now = Instant.now();
    return candidates.stream()
        .map(c -> c.withScore(scorer.score(
            /*semantic*/ 1.0 - c.cosineDistance(),
            /*recency */ c.lastSeen(),
            /*freq    */ c.accessCount()))
        .sorted(comparingDouble(ScoredChunk::score).reversed())
        .limit(q.k())
        .toList();
}
```



```
-- CANDIDATE_SQL : over-fetch by cosine distance, carry the signals for Java re-ranking.
-- '<=>' is pgvector's cosine-distance operator (smaller = closer).
SELECT id,
        content,
        metadata,
        (metadata->>'access_count')::int           AS access_count,
        (metadata->>'last_seen')::timestampz       AS last_seen,
        embedding <=> CAST(? AS vector)          AS cosine_distance
FROM vector_store
WHERE metadata->>'session_id' = ?                -- session-scoped retrieval
AND metadata->>'type' = ?                        -- 'chat_message' | 'deployment' | 'log' | 'kafka'
ORDER BY embedding <=> CAST(? AS vector)
LIMIT ?;                                       -- k*4
```

```
// RecencyFrequencyScorer — pure, unit-testable
double score(double semantic, Instant lastSeen, int accessCount) {
    double ageDays = Duration.between(lastSeen, Instant.now()).toHours() / 24.0;
    double recency = Math.exp(-RECENCY_LAMBDA * ageDays); //  $\lambda \approx 0.05 \rightarrow \sim 14$ -day half-life
    double frequency = Math.log1p(accessCount) / Math.log1p(FREQ_NORM); // saturating,
    FREQ_NORM  $\approx 20$ 
    return 0.35 * semantic + 0.35 * recency + 0.30 * Math.min(frequency, 1.0);
}
```

"Related past sessions" is the same query with the session_id filter removed (or widened to all sessions), capped at a smaller k. Increment access_count and refresh last_seen on the rows you actually inject — that is what makes the frequency signal meaningful over time.

3.7 Token estimation

Don't hardcode a tokenizer to one model — Ollama models vary. Wrap Spring AI's TokenCountEstimator where available, and fall back to a chars/4 heuristic for budgeting triggers. Budget arithmetic only needs to be *approximately* right to drive the trigger; exactness isn't required.

4. Spring AI architecture — detailed component design

4.1 ChatClientConfig (@Configuration)

Owns the wiring. Beans:

```

@Configuration
class ChatClientConfig {

    @Bean ChatMemoryRepository chatMemoryRepository(JdbcTemplate jdbc) {
        return JdbcChatMemoryRepository.builder().jdbcTemplate(jdbc).build();
    }

    @Bean ChatMemory chatMemory(ChatMemoryRepository repo) {
        return MessageWindowChatMemory.builder()
            .chatMemoryRepository(repo)
            .maxMessages(8)           // last 4 turns verbatim
            .build();
    }

    @Bean ChatClient.Builder chatClientBuilder(ChatModel chatModel) { // base; ModelRouter rebinds
model
        return ChatClient.builder(chatModel)
            .defaultSystem(SYSTEM_PROMPT)
            .defaultAdvisors(/* context + RAG advisors injected by order */);
    }

    @Bean ToolCallingManager toolCallingManager() { // for manual/confirmed execution (§5)
        return ToolCallingManager.builder().build();
    }
}

```

4.2 ModelRouter — dynamic runtime/model switching

Because users can deploy several Ollama containers and pull several models, build the ChatClient per request from the base builder, rebinding the model via options:

```

@Service
class ModelRouter {
    ChatClient clientFor(String runtimeId, String modelId) {
        var runtime = modelRuntimeService.get(runtimeId); // base-url of THAT Ollama container
        var ollama = OllamaChatModel.builder()
            .ollamaApi(OllamaApi.builder().baseUrl(runtime.baseUrl()).build())
            .defaultOptions(OllamaOptions.builder().model(modelId).build())
            .build();
        return ChatClient.builder(ollama).defaultSystem(SYSTEM_PROMPT).build();
    }
}

```

This is the "model router pattern" from your request — concrete, and it naturally extends to DMR by swapping the ChatModel implementation while keeping clientFor(...) identical.

4.3 RagChatService — sessions, assembly, streaming

```

@Service
class RagChatService {
    Flux<ServerSentEvent<ChatToken>> stream(String sessionId, String userMsg, ModelSelection sel) {
        var client = modelRouter.clientFor(sel.runtimeId(), sel.modelId());
        return client.prompt()
            .user(userMsg)
            .advisors(a -> a.param(ChatMemory.CONVERSATION_ID, sessionId)) // window keyed by session
            .toolCallbacks(infrastructureToolCallbacks) // see §4.4 / §5
            .stream()
            .chatResponse()
            .map(this::toSse)
            .doOnComplete(() -> summaryService.summariseIfNeeded(sessionId)); // async compress
    }
}

```

SSE + WebFlux — a grounded recommendation. Your app is spring-boot-starter-web (servlet) + JPA. Do **not** bolt on WebFlux + R2DBC for streaming; that doubles the web stack and fights your JPA persistence. Spring MVC can return Flux<ServerSentEvent<...>> directly from a @GetMapping(produces = MediaType.TEXT_EVENT_STREAM_VALUE) and will stream it over the servlet container's async I/O. Keep one stack. Apply onBackpressureBuffer and a sane timeout on the Flux. This is the right call for a memory-conscious local app and avoids a large, risky refactor.

4.4 InfrastructureTools (@Component, @Tool methods)

The agent's hands. Each method is a thin wrapper over existing services — **no new business logic in the tool layer**. Register via MethodToolCallbackProvider.builder().toolObjects(infraTools).build().

```

@Component
class InfrastructureTools {
    // — Read-only: never gated

    @Tool(description = "List all deployed instances with status, type, and host port")
    List<InstanceSummary> listInstances() { return dbInstanceService.listAll().stream()...; }

    @Tool(description = "Read the last N log lines from an instance by name")
    String readLogs(String instanceName, int lines) { ... dbInstanceService.getLogs(id, lines); }

    @Tool(description = "Generate a connection string / Spring config for an instance")
    String connectionConfig(String instanceName) { ... }

    // — Writes: confirmation-gated via manual execution (see §5) —
    @Tool(description = "Deploy a new database/service container")
    DeploymentSummary deployDatabase(String type, String version, String name, Integer hostPort) { ... }

    @Tool(description = "Create a Kafka topic on a deployed Kafka instance")
    TopicSummary createKafkaTopic(String instanceName, String topic, int partitions, int rf) { ... }

    @Tool(description = "Pull a model into a deployed LLM runtime")
    ModelPullSummary pullModel(String runtimeName, String modelTag) { ... }

    // — Destructive: always confirmation-gated

    @Tool(description = "Stop a running instance") InstanceSummary stopInstance(String name) { ... }
    @Tool(description = "Remove an instance and its data") void removeInstance(String name) { ... }
}

```

Because these wrap [DbInstanceService](#) ([../backend/src/main/java/com/dbdeployer/service/DbInstanceService.java](#)) and the future KafkaAdminService/ModelRuntimeService, the agent executes the **same code paths as the UI** — same validation (DeploymentValidations), same pipeline, same recovery. That is the single most defensible thing in the whole design.

4.5 Advisor chain — final order

```

[order 100] ContextAssemblyAdvisor → injects rolling summary + retrieved memories (§3)
[order 200] InfraRagAdvisor       → QuestionAnswerAdvisor over the infra knowledge base (§ Phase 4)
[order 300] (built-in) MessageChatMemoryAdvisor → verbatim window from RollingWindowChatMemory
[auto/last] ToolCallingAdvisor   → only when running in auto-exec mode; DISABLED for write tools (§5)

```

Memory + RAG advisors deliberately sit *before* tool calling so the model sees full context when it decides whether to call a tool.

5. MemPalace integration assessment

Verdict: do not take a runtime dependency on MemPalace. Borrow its *structure* as inspiration now; support it as an *opt-in, feature-flagged MCP client* in Phase 5. Never make it part of the default install.

5.1 Why not embed it

- **It breaks zero-extra-service.** MemPalace is a Python process backed by **ChromaDB**. Adopting it as core means a second runtime *and* a second vector store running on the developer's laptop — precisely the principle you set out to protect. You already have pgvector inside the Postgres you provision.
- **pgvector + §3 already covers the need.** MemPalace's headline is long-horizon recall (96.6% R@5 on LongMemEval). That benchmark stresses a *general personal-memory* product across thousands of unrelated conversations. Port Wrangler's memory is **scoped to infra operations on one machine** — the retrieval surface is deployment records, logs, topic metadata, and this user's own chat history. The "related past sessions" layer in §3.6 (cross-session vector search) already gives you cross-conversation memory at this scale. The recall pressure that justifies MemPalace's complexity simply isn't present here.
- **Resume math:** "I added a 768-dim pgvector retrieval layer with recency/frequency re-ranking and rolling summarisation" is a stronger, more *ownable* interview story than "I shelled out to a third-party memory service." You want to demonstrate you can *build* the memory system, not glue one in.

5.2 Why it's still worth a small, optional hook

Here's the elegant part: **Port Wrangler deploys containers for a living.** So the lowest-cost way to "support" MemPalace is to let the tool deploy it like anything else, and let the assistant use it *if the user opted in* — via Spring AI's MCP client. That demonstrates MCP-client integration (a genuine resume signal) without burdening the default install.

Concretely, a Phase-5 stretch:

1. Add a catalog entry so MemPalace is a deployable managed container (a `RuntimeType.MEM_PALACE`), reusing the exact `DeployStep` pipeline. Users who want it can one-click it.
2. Wire `spring-ai-starter-mcp-client` pointed at the MemPalace MCP server (its 29 tools). Spring AI auto-registers MCP tools as `ToolCallbacks`, so they appear to the agent alongside `InfrastructureTools`.
3. Gate the whole thing behind `portwrangler.memory.mcp.enabled=false` (default off).

```
# application.yml — only active when the user enables it
portwrangler:
  memory:
    mcp:
      enabled: false
spring:
  ai:
    mcp:
      client:
        sse:
          connections:
            mempalace:
              url: http://localhost:8077 # the MemPalace container the user deployed
```

Net: **inspiration now, optional MCP integration as a Phase-5 stretch, never a hard dependency.**

6. Phased implementation roadmap

Six phases, ~22 weeks part-time (saved 3 weeks by extracting Kafka management to a separate project). Each phase lists concrete deliverables (files/classes/tests), the resume signal, and the top risk with its mitigation. Phases are grounded in the real packages.

Phase 0 — Hardening (Weeks 1–2)

Deliverables

- **Finish the Liquibase cutover** (not Flyway — Correction #1): generate a baseline changeset from the current Hibernate-built schema, uncomment `spring.liquibase` ([application.yml:39](#) [\(..../backend/src/main/resources/application.yml#L39\)](#)), set `ddl-auto: validate`. (Full plan in §7.)
- **CI:** `.github/workflows/ci.yml` — `./gradlew build spotlessCheck`, JDK 21, Testcontainers enabled. You already enforce `googleJavaFormat` via Spotless; make CI fail on format drift.
- **First integration test of the existing pipeline:** `DeployPipelineIT` using Testcontainers `PostgreSQLContainer` for the system DB and a real Docker socket (the runner has one) to deploy a throwaway `redis:7`, asserting the `deployment_pipeline + pipeline_step` rows transition `PENDING→RUNNING→SUCCESS`. This locks in the behaviour before AI code lands.

Resume signal: disciplined migration management, Testcontainers, reproducible CI.

Risk: the `com.dbdeployer` → `io.portwrangler` package rename is tempting but churns all ~85 files and every import. **Mitigation:** **don't**. Rename the *product/artifact/repo* ("Port Wrangler") and leave the Java package `com.dbdeployer` — package names are invisible to users and the rename buys nothing but merge pain. Spend the budget on tests instead.

Phase 1 — Quick wins + Docker Compose export (Weeks 3–4)

Kafka management is a separate project. Port Wrangler still *deploys* Kafka (`DbType.KAFKA`, controller port already wired in [DockerDeployEngine:202](#) [\(..../backend/src/main/java/com/dbdeployer/deploy/DockerDeployEngine.java#L202-L206\)](#)) and the agent tool layer retains a thin `createKafkaTopic(...)` call for agent-initiated topic creation. Full topic CRUD, consumer-group lag, message browser, and Schema Registry belong in the dedicated Kafka tool — not here.

This phase uses the 3 weeks saved from dropping Kafka management to ship high-visibility quick wins while the codebase is still simple (before Spring AI lands).

Deliverables

Docker Compose export

- `api/ComposeExportController.java` — `GET /api/export/docker-compose` walks all non-REMOVED, non-system `DeploymentConfig` rows and emits a valid `docker-compose.yml`.
- `service/ComposeExportService.java` — maps each `DeploymentConfig` to a Compose service block (image, ports, environment, volumes, restart policy) using the same data `createContainer` already uses. ~80 lines of code, zero new dependencies.

- Handles multi-port services (Neo4j bolt, ClickHouse HTTP) via the same special-case logic in DockerDeployEngine.
- Frontend: a single "Export as docker-compose.yml" button on the instances list, triggering a file download.
- **Tests:** assert the YAML output for a known DeploymentConfig round-trips to valid Compose.

Connection string + Spring config bootstrap

- Extend the existing ConnectionStringBuilder with a springBootProperties(config) method that returns a ready-to-paste application.properties block for each instance type (JDBC URL, username, password, driver class, Spring AI Ollama base-url, etc.).
- Surface this in the UI as a "Copy Spring config" button per instance — using data already in the DB.
- Zero new dependencies; ~40 lines per service type.

Resume signal: high developer-experience signal per line of code; Docker Compose export is the most-starred feature request in every similar tool's issue tracker.

Risk: Compose export produces technically valid YAML but the user's compose version may differ.

Mitigation: pin version: "3.9" and document that the output is a starting point, not a production spec.

Phase 2 — LLM Runtime Manager + Model Cookbook (Weeks 5–8)

Deliverables — Runtime Manager

- runtime/ModelRuntime.java (enum: OLLAMA, DOCKER_MODEL_RUNNER) + catalog entries reusing the DatabaseCatalog.DbDefinition shape (image, default port, data volume, env). Ollama → ollama/ollama:latest, port 11434, volume /root/.ollama.
- **GPU detection without shelling out** (runtime/GpuDetector.java): inspect Docker via the SDK (docker.infoCmd() exposes runtimes — presence of the nvidia runtime ⇒ NVIDIA toolkit), probe device files for AMD (/dev/kfd, /dev/dri), read OS/arch for Apple Silicon (os.arch == aarch64 + macOS). Never run nvidia-smi from Java on the host; if you need in-container confirmation, reuse the existing safe execCapture(...) (no shell, argv array).
- **GPU-aware container config** — extend the HostConfig you already build in createContainer:
 - NVIDIA: .withDeviceRequests(List.of(new DeviceRequest().withDriver("nvidia").withCount(-1).withCapabilities(List.of(List.of("gpu")))))
 - AMD ROCm: .withDevices(new Device("rwm", "/dev/kfd", "/dev/kfd"), new Device("rwm", "/dev/dri", "/dev/dri")) + video/render groups
 - Apple Silicon **containerised: no GPU** — Docker Desktop's Linux VM has no Metal passthrough; detect and surface "this Ollama will run CPU-only; for GPU use native Ollama" (Correction #6).
 - CPU fallback: usable for ≤3–4B models; warn on 7B+ (multi-second/token on laptop CPU).
- runtime/ModelPullStep.java — a new DeployStep/async job that calls Ollama's pull API and streams progress into a pipeline_step row, exactly like ImagePullStep streams image-pull progress.
- Model metrics polling + runtime/SpringAiSnippetService.java (generate a ready-to-paste Spring AI application.yml + dependency block for the deployed runtime+model).
- **Tests:** unit-test GpuDetector against captured docker info JSON fixtures (no GPU needed in CI).

Deliverables — Model Cookbook (inspired by Odysseus; ~2 extra days)

The Model Cookbook answers "what can my machine actually run?" using hardware the GpuDetector already reads — no extra detection work needed.

- runtime/SystemProfile.java — aggregates GpuDetector output + OS RAM/CPU into one record: gpuVendor, vramMb, totalRamMb, cpuCores, platform, isContainerGpu.
- runtime/ModelCatalog.java — a static map of curated Ollama models (same pattern as DatabaseCatalog):

```
record ModelDefinition(  
    String ollamaTag,      // "llama3.1:8b"  
    String family,        // "Llama 3.1"  
    ModelType type,       // CHAT, CODE, EMBEDDING, VISION, REASONING  
    long paramsBillions,  
    Quantization defaultQuant,  
    long minVramMb,       // Q4_K_M 8B ≈ 5500 MB  
    long minRamMb,        // CPU fallback requirement  
    String description  
) {}
```

Seed with ~25 models: llama3.1 (8B/70B), mistral-nemo, gemma2 (2B/9B), qwen2.5 (7B/14B/72B), phi3.5-mini, codestral, deepseek-coder-v2, llava, deepseek-r1 (7B/32B), nomic-embed-text, mx-bai-embed-large. Refresh via PR — no external API call at runtime.

- runtime/ModelSuggestionService.java — scores each catalog entry against the system profile:

```
CompatibilityLevel score(ModelDefinition m, SystemProfile sys) {  
    long effectiveVram = sys.isAppleSilicon() ? sys.totalRamMb() : sys.vramMb();  
    if (effectiveVram >= m.minVramMb() * 1.3) return FAST;      // GPU, comfortable headroom  
    if (effectiveVram >= m.minVramMb())      return OK;        // GPU, tight  
    if (sys.totalRamMb() >= m.minRamMb())    return CPU_ONLY;  // slow but runnable  
    return TOO_LARGE;  
}
```

- api/ModelCookbookController.java — GET /api/models/suggestions?
type=CHAT&compat=FAST,OK
returns ranked suggestions with compatibility level, estimated token speed tier, and a one-click pull endpoint.
- Frontend: filter chips (All · Chat · Code · Embedding · Vision · Reasoning) ×
(All · Fast · OK · CPU only) + model cards with a green/amber/red compatibility badge and a "Pull" button that fires the ModelPullStep pipeline.

Resume signal: hardware-aware model routing + the Model Cookbook UX is the single most impressive demo frame in the whole product — a grid of model cards colour-coded to the user's GPU.

Risk: GPU permutations are unbounded and untestable in CI. **Mitigation:** fixture-driven unit tests for GpuDetector and ModelSuggestionService; treat GPU container config as best-effort with a clear CPU fallback.

Phase 3 — Spring AI foundation + Model Comparison (Weeks 9–11)

Deliverables — Spring AI foundation

- build.gradle.kts: BOM 1.0.3 + spring-ai-starter-model-ollama (Appendix).

- ai/ChatClientConfig.java, ai/ModelRouter.java, ai/RagChatService.java (§4).
- api/ChatController.java — @GetMapping(produces=TEXT_EVENT_STREAM_VALUE) returning Flux<ServerSentEvent<...>> on the existing servlet stack (§4.3 — no WebFlux).
- MessageWindowChatMemory + JdbcChatMemoryRepository (SPRING_AI_CHAT_MEMORY table, §7).
- Dynamic model switching via ModelRouter; the static system prompt.
- **Tests:** @SpringBootTest with a stub ChatModel returning canned ChatResponses — assert SSE framing and that the conversation id flows to memory.

Deliverables — Model Comparison (inspired by Odysseus; ~1.5 extra days)

Once ModelRouter exists, running the same prompt against two models simultaneously is Flux.merge over two ChatClient streams — the infrastructure is already there.

- api/ModelComparisonController.java — GET /api/models/compare accepts prompt, modelA, modelB (optionally modelC) and streams responses for all concurrently via SSE, each tagged with its model id.
- service/ModelComparisonService.java — builds one ChatClient per model via ModelRouter, merges the Flux streams:

```
Flux<ComparisonChunk> compare(CompareRequest req) {
    return Flux.merge(
        modelRouter.clientFor(req.getRuntimeId(), req.modelA())
            .prompt(req.prompt()).stream().chatResponse()
            .map(r -> new ComparisonChunk("A", req.modelA(), r.getResult().getOutput().getText())),
        modelRouter.clientFor(req.getRuntimeId(), req.modelB())
            .prompt(req.prompt()).stream().chatResponse()
            .map(r -> new ComparisonChunk("B", req.modelB(), r.getResult().getOutput().getText()))
    );
}
```

- Frontend: a split-pane view — two (or three) chat columns streaming in parallel, labelled with model name + token speed. No model wins selected by default; the developer judges.
- Optional: a "blind mode" toggle that hides model names until the user picks a preferred response (the Odysseus "blind testing" concept, scoped to models you already manage).
- **Tests:** stub two ChatModel beans, assert both SSE streams are tagged correctly and the merged flux completes when both finish.

Resume signal: Spring AI 1.0 GA, SSE streaming, concurrent multi-model inference — three strong talking points in one phase. The Model Comparison UI is an immediately understandable demo moment.

Phase 4 — RAG pipeline (Weeks 12–15)

Deliverables

- Liquibase V8__enable_pgvector.sql (CREATE EXTENSION IF NOT EXISTS vector) + spring-ai-starter-vector-store-pgvector; PgVectorStore configured HNSW / cosine / 768 (§7).
- rag/IngestionService.java — index **deployment records** (one doc per instance: type, version, ports, env summary), **container logs** (via existing DockerDeployEngine.getLogs), **Kafka topic metadata**, and **model info**. Re-index on deploy/remove and on a schedule.
- **Log chunking strategy** (logs are not prose): split on log-record boundaries (timestamp-prefixed lines), group N adjacent records per chunk, and **promote severity into metadata** (level=ERROR|WARN|INFO). Store level so retrieval can boost it.
- **Noise handling:** add a metadata-weighted term to retrieval so ERROR/WARN chunks outrank

INFO for the same semantic distance (extend the §3.6 scorer with a small `severity_boost`).

- ContextAssemblyAdvisor + the §3 token-optimization classes land here (the chat from Phase 3 gets RAG-augmented and summary-compressed).
- QuestionAnswerAdvisor over the infra KB with **metadata filtering** (filter by `instance_type`, `level`, `date range`) using `FilterExpressionBuilder`.
- **Tests:** ingest a known log blob, assert an ERROR chunk is retrieved above INFO for a generic query.

Resume signal: end-to-end RAG, domain-specific chunking, metadata filtering, the token-savings algorithm — this is the centrepiece.

Risk: embedding everything bloats the vector table and slows ingestion. **Mitigation:** cap log ingestion to last-N lines per instance, dedupe identical stack traces, and TTL old log vectors.

Phase 5 — Agentic assistant + safety + MCP server (Weeks 16–19)

Deliverables — InfrastructureTools + safety

- `ai/InfrastructureTools.java` (§4.4) registered via `MethodToolCallbackProvider`.
- **Confirmation flow via manual tool execution** — the core safety design. Read-only tools run auto; **write/destructive tools run only after explicit user approval**, using Spring AI's user-controlled execution (disable internal execution, drive the loop yourself):

```
// Write/destructive path: do NOT let the advisor auto-execute.
var options = ToolCallingChatOptions.builder()
    .toolCallbacks(ToolCallbacks.from(infrastructureTools))
    .internalToolExecutionEnabled(false)    // <-- the key flag
    .build();
var prompt = new Prompt(messages, options);
ChatResponse resp = chatModel.call(prompt);

while (resp.hasToolCalls()) {
    for (var call : resp.getToolCalls()) {
        if (isDestructive(call.getToolName())) {
            emitConfirmationRequest(call.getToolName(), call.getArguments());
            if (!awaitUserApproval(call)) { break; }
        }
    }
    ToolExecutionResult r = toolCallingManager.executeToolCalls(prompt, resp);
    prompt = new Prompt(r.conversationHistory(), options);
    resp = chatModel.call(prompt);
}
```

- **Classification:** read-only (`listInstances`, `readLogs`, `stackSummary`) → never gated; writes (`deployDatabase`, `pullModel`, `createKafkaTopic`) → confirm; destructive (`stopInstance`, `removeInstance`) → confirm with target echoed back.
- **"What's my stack" tool** (`stackSummary`) — a `@Tool(returnDirect = true)` method that calls `dbInstanceService.listAll()` + `connectionStringBuilder` to return a structured summary of every running instance (type, version, port, connection string, status). The agent answers "what's running locally?" in one tool call; the UI surfaces this as a "Copy as .env" / "Copy as Spring config" button. Zero new data fetching — reuses what already exists.
- **Read-only mode:** `portwrangler.agent.read-only=true` flag (and UI toggle) strips all non-read `ToolCallbacks` so write tools simply don't exist in that context, rather than being refused.

- **Runaway-loop guard:** cap tool iterations per turn (MAX_TOOL_ROUNDS = 6); surface "agent exceeded tool budget" if exceeded.
- **Tool-call visibility:** stream each tool call (name + arguments + result summary) as its own SSE event for a live "what the agent is doing" trace.
- **(Stretch)** optional MemPalace MCP client (§5.2), flag-gated.
- **Tests:** assert removeInstance does **not** execute without approval; assert read-only mode removes write tools; assert the loop cap fires.

Deliverables — Port Wrangler MCP server (inspired by Odysseus; ~2 days with the Spring AI starter)

This is the highest-leverage new feature in the roadmap. Port Wrangler exposes its own MCP server so external tools — **Cursor**, **Continue.dev**, **VS Code Copilot extensions**, **Claude Desktop** — can call Port Wrangler's infrastructure tools directly from inside the IDE, without opening Port Wrangler.

IDE AI assistant → MCP → **Port Wrangler MCP server** → DbInstanceService / ModelRuntimeService

- Add spring-ai-starter-mcp-server (see Appendix B).
- mcp/PortWranglerMcpServer.java — annotates the same InfrastructureTools methods as MCP tools via @McpTool. The tool definitions are *identical* to the agent tools — same validation, same pipeline, no duplication.
- Expose on a configurable port (default 8081) so it doesn't conflict with the main API (8080).
- The MCP server runs read-only by default; users opt into write tools via config.
- **mcp/McpServerConfig.java:**

```
@Configuration
class McpServerConfig {
    @Bean McpServer mcpServer(InfrastructureTools tools) {
        return McpServer.builder()
            .name("port-wrangler")
            .version("1.0.0")
            .tools(MethodToolCallbackProvider.builder().toolObjects(tools).build())
            .build();
    }
}
```

- **.mcp/port-wrangler.json** (checked in at repo root) — the MCP client config users paste into Cursor / Claude Desktop:

```
{
  "mcpServers": {
    "port-wrangler": {
      "url": "http://localhost:8081/sse",
      "description": "Port Wrangler local infra tools"
    }
  }
}
```

- **Tests:** start the MCP server, call listInstances via MCP client, assert the response shape.

Resume signal: building *and exposing* an MCP server is rare in Java portfolios. "Cursor can deploy your Postgres via Port Wrangler" is a demo that writes itself.

Risk: MCP server exposes destructive tools to any client that connects. **Mitigation:** read-only by default; write tools only enabled via explicit `portwrangler.mcp.write-enabled=true`; bind to localhost only.

Phase 6 — Polish & launch (Weeks 20–22)

Docs site, demo video, verified native installers, README overhaul, community launch. Fully detailed in §8.

Resume signal: shipped, documented, launched OSS with traction.

Risk: scope creep delays launch indefinitely. **Mitigation:** launch at "Phase 4 complete + basic agent" if needed; the agent can mature in public.

7. Database schema — final version

7.1 The cutover that must happen first

The other docs skipped this because they didn't know it was needed. Today the schema is managed by **Hibernate ddl-auto: update**, with Liquibase present but dormant. You cannot add vector/AI tables cleanly on top of update (it will fight your changesets and silently re-derive columns). So Phase 0 must close the loop you already started:

1. **Baseline** — capture the *current* live schema as `V5__baseline.sql` (hand-write it, or `./gradlew ... liquibase diffChangeLog` against the running DB), so the drift in §0 Correction #1 is reconciled (the entities, e.g. `DeploymentPipeline.deployment_container_id / is_template`, become the source of truth).
2. **Enable Liquibase** — uncomment [application.yml:39-40](#) ([../backend/src/main/resources/application.yml#L39-L40](#)) (`spring.liquibase.change-log: classpath:db/changelog/db.changelog-master.yaml`).
3. **Flip Hibernate to read-only DDL** — `ddl-auto: validate` (validate, don't mutate).
4. From here, **every** schema change is a changeset. Keep the existing convention exactly: `V<N>__<snake_desc>.sql`, first line `--liquibase formatted sql`, then `--changeset portWrangler:<n> labels:<phase> comment:<...>`, registered in `db.changelog-master.yaml`.

7.2 New changesets by phase

File	Phase	Purpose
<code>V5__baseline.sql</code>	0	Reconcile live schema; stop <code>ddl-auto: update</code>
<code>V6__add_model_runtime.sql</code>	2	<code>model_runtime</code> , <code>pulled_model</code>
<code>V7__add_chat_memory.sql</code>	3	<code>SPRING_AI_CHAT_MEMORY</code> , <code>chat_session</code> , <code>chat_message</code>
<code>V8__enable_pgvector.sql</code>	4	<code>CREATE EXTENSION vector</code> ; <code>vector_store</code> + HNSW/GIN indexes

kafka_consumer_lag_sample belongs in the separate Kafka tool's schema, not here.

7.3 DDL (the AI/Kafka tables)

```

--liquibase formatted sql
--changeset portWrangler:7 labels:phase3 comment:Chat memory + sessions

-- Spring AI JdbcChatMemoryRepository default table (verbatim window storage).
CREATE TABLE SPRING_AI_CHAT_MEMORY (
  conversation_id VARCHAR(36) NOT NULL,
  content TEXT NOT NULL,
  type VARCHAR(10) NOT NULL, -- USER | ASSISTANT | SYSTEM | TOOL
  "timestamp" TIMESTAMPTZ NOT NULL,
  CONSTRAINT chk_sacm_type CHECK (type IN ('USER','ASSISTANT','SYSTEM','TOOL'))
);
CREATE INDEX idx_sacm_conv_time ON SPRING_AI_CHAT_MEMORY (conversation_id, "timestamp");

-- Session + rolling-summary state (drives §3 token optimization).
CREATE TABLE chat_session (
  id VARCHAR(36) NOT NULL PRIMARY KEY,
  title VARCHAR(255),
  system_prompt TEXT,
  model_runtime_id VARCHAR(36), -- FK to model_runtime (nullable: default runtime)
  model_id VARCHAR(255),
  rolling_summary TEXT,
  summary_token_count INT NOT NULL DEFAULT 0,
  summarized_through_seq INT NOT NULL DEFAULT 0,
  current_seq INT NOT NULL DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ
);

-- Per-message store with retrieval signals (access_count/last_seen feed §3.6 scoring).
CREATE TABLE chat_message (
  id VARCHAR(36) NOT NULL PRIMARY KEY,
  session_id VARCHAR(36) NOT NULL REFERENCES chat_session(id),
  seq INT NOT NULL,
  role VARCHAR(10) NOT NULL,
  content TEXT NOT NULL,
  token_count INT NOT NULL DEFAULT 0,
  access_count INT NOT NULL DEFAULT 0,
  last_seen TIMESTAMPTZ,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  CONSTRAINT uq_chat_message_seq UNIQUE (session_id, seq)
);
CREATE INDEX idx_chat_message_session_seq ON chat_message (session_id, seq);

```

```
--liquibase formatted sql
--changeset portWrangler:8 labels:phase4 comment:pgvector + shared vector store
--changeset attribute runOnChange:false

CREATE EXTENSION IF NOT EXISTS vector;

-- ONE shared store for ALL embeddings (chat messages, deployments, logs, kafka metadata),
-- discriminated by metadata->'type'. Matches Spring AI PgVectorStore's default table so the
-- starter can manage it; one HNSW index instead of N.
CREATE TABLE vector_store (
  id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  content TEXT,
  metadata JSONB,
  embedding VECTOR(768)           -- nomic-embed-text; FIXED at create time (§2.3)
);

-- HNSW: better query performance than IVFFlat AND builds on an empty table (no training step) —
-- ideal for a fresh local install. Cosine matches Spring AI's default distance.
CREATE INDEX idx_vector_store_hnsw ON vector_store
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

-- GIN on metadata so metadata filtering (type, session_id, instance_type, level) is indexed.
CREATE INDEX idx_vector_store_metadata ON vector_store USING gin (metadata jsonb_path_ops);
```

```
--liquibase formatted sql
--changeset portWrangler:6 labels:phase2 comment:LLM runtimes + pulled models

CREATE TABLE model_runtime (
  id          VARCHAR(36) NOT NULL PRIMARY KEY,
  runtime_type VARCHAR(40) NOT NULL,           -- OLLAMA | DOCKER_MODEL_RUNNER
  config_id   VARCHAR(36) REFERENCES deployment_config(id), -- the managed container
  base_url    VARCHAR(255) NOT NULL,
  gpu_vendor  VARCHAR(20),                     -- NVIDIA | AMD | APPLE | NONE
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE pulled_model (
  id          VARCHAR(36) NOT NULL PRIMARY KEY,
  runtime_id  VARCHAR(36) NOT NULL REFERENCES model_runtime(id),
  model_name  VARCHAR(255) NOT NULL,           -- e.g. llama3.1:8b
  size_bytes  BIGINT,
  quantization VARCHAR(40),
  digest      VARCHAR(255),
  pulled_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_used_at TIMESTAMPTZ,
  CONSTRAINT uq_pulled_model UNIQUE (runtime_id, model_name)
);
```

7.4 Non-obvious decisions, justified

- **One shared vector_store, not per-domain tables.** A single HNSW index is cheaper to maintain than four, and it matches Spring AI's PgVectorStore default so the starter can own the table.

Cross-domain isolation is achieved with metadata->>'type' + the GIN index, not separate tables.

- **HNSW over IVFFlat.** IVFFlat needs representative data to train its lists; on a fresh install the table is empty, so IVFFlat would build a bad index (or you'd defer index creation). HNSW has no training step, builds on an empty table, and gives better recall/latency — at the cost of slightly higher memory, which is negligible at local scale.
- **Embedding dim hardcoded to 768 in DDL.** This is deliberate, not lazy: a pgvector column dimension is immutable. Pinning it documents the model contract; switching embedding models is an explicit migration (drop column + re-embed), never a silent config change (§2.3).
- **chat_message duplicates content that also lives in SPRING_AI_CHAT_MEMORY.** That's intentional separation of concerns: the Spring AI table is the framework-owned verbatim *window*; chat_message is your *retrievable corpus* with scoring signals (access_count, last_seen, seq). They serve different layers of §3.1 and shouldn't be conflated.

8. Open-source launch plan (30 days)

8.1 README — win the first 10 seconds

Order matters; a visitor decides in one scroll.

1. **One-line hero + animated GIF above the fold.** Hero: *"Deploy Postgres, Kafka, and a local LLM — and an AI agent that operates all of them — from one local app."* The GIF (§8.2) auto-plays the wow.
2. **Badges:** build, license, latest release, "26 services supported", Java 21 / Spring Boot.
3. **30-second quickstart:** download the DMG/EXE (you already build them) *or* docker compose up. Two paths, both real.
4. **Feature matrix** (databases / Kafka / LLMs / AI agent) with a single screenshot each.
5. **"vs the alternatives" table** — a trimmed §1.1. Honesty about scope builds trust.
6. **Architecture diagram** — the pipeline + Spring AI + pgvector. Reuse backend/docs/*.mermaid.
7. CONTRIBUTING / good-first-issues link last.

8.2 Demo content

One 60–90s screen capture, exported as both an MP4 (for HN/PH) and a sub-10MB GIF (for the README):
deploy Postgres → deploy Kafka → create topic orders (6 partitions) in the UI → deploy Ollama + pull llama3.1:8b → open chat → type *"deploy a Redis named cache and give me the Spring config"* → agent proposes deployDatabase, you click **Confirm**, it deploys and prints the config. That single take demonstrates all four lanes *and* the agentic seam — it is the entire pitch in 90 seconds.

8.3 Community posts (sequence + timing, US/ET)

Day	Channel	Angle
1	r/selfhosted, r/java	"I built a local infra cockpit (DB + Kafka + LLM) in Spring Boot" + GIF
3	r/LocalLLaMA	LLM-runtime + GPU-detection + Spring AI agent angle
7	Show HN (Tue–Thu, ~9am ET)	"Show HN: Port Wrangler — local infra + an AI agent that runs it" Deep-dive: the token-optimization algorithm (§3) — your most technical post
8	dev.to / Medium	

10	r/devops, r/SpringBoot	The @Tool-over-services agentic design
14	Product Hunt (Tue, 12:01am PT)	Rally the stars you've accumulated
21	Spring community / Spring AI showcase	Submit as a Spring AI reference app

Don't fire them all at once — each gets its own front-page shot; space them so one can recover from a flop.

8.4 First 10 "good first issues"

Scoped so a newcomer can ship in an evening, and each maps to a real extension point:

1. Add a new DbType + DatabaseCatalog entry (e.g. ScyllaDB) — pure pattern-follow.
2. Add a connection-string format in ConnectionStringBuilder for an existing type.
3. Frontend empty-state for the instances list when nothing is deployed.
4. A Testcontainers test for one lifecycle path (stopInstance → RUNNING→STOPPED).
5. Add a model-catalog entry (a new Ollama model card) once Phase 2 lands.
6. Surface a Kafka topic config field (retention.ms) in the topic-create form.
7. Keyboard a11y pass on the deploy modal.
8. A GpuDetector fixture test for a new docker info shape.
9. --read-only / config flag to launch the agent in read-only mode (Phase 5).
10. Docs: a per-database "connect with your favourite client" snippet page.

8.5 Week-by-week toward the first 100 stars

- **Week 1:** README overhaul + GIF + 2 native installer releases (Mac/Win) attached to a GH Release.
Soft posts (r/selfhosted, r/java). Target: 25 stars.
- **Week 2:** Show HN + the §3 deep-dive blog. The technical post is what converts skeptics. Target: 60.
- **Week 3:** Product Hunt + r/devops/r/SpringBoot. Respond to every comment within the hour. Target: 85.
- **Week 4:** Spring AI showcase submission + ship 2 good-first-issue PRs from contributors (visible activity compounds). Target: 100+.

The lever is **the demo GIF and the token-optimization write-up**, not the post count. Depth travels.

Appendix A — Consolidated corrections (for the other docs' authors)

Prior claim	Status	Grounded correction
Use Flyway	x wrong tool	Liquibase is already on the classpath; finish the cutover (§7.1)
Spring AI 1.0.0-M5	x violates constraint	Pin 1.0.3 GA
"14 database types"	x undercount	26 DbType entries; it's a general infra deployer
"Build a 4-step pipeline"	x already done	Reuse the DeployStep SPI
MCP starter ...-spring-boot-starter	x stale naming	spring-ai-starter-mcp-client
Apple Silicon containers get		

Metal	x false	Dockerised Ollama on macOS is CPU-only
ddl-auto already replaced	x not in runtime	ddl-auto: update is still active in application.yml

Appendix B — Exact dependency block (Gradle Kotlin DSL, Spring AI 1.0.3 GA)

```
dependencies {  
    // Spring AI BOM — pins all Spring AI artifacts to the 1.0.x GA line  
    implementation(platform("org.springframework.ai:spring-ai-bom:1.0.3"))  
  
    // Ollama: chat + embeddings in one starter  
    implementation("org.springframework.ai:spring-ai-starter-model-ollama")  
  
    // pgvector vector store (extension on your existing Postgres 16)  
    implementation("org.springframework.ai:spring-ai-starter-vector-store-pgvector")  
  
    // JDBC-backed chat memory window (verbatim turns)  
    implementation("org.springframework.ai:spring-ai-starter-model-chat-memory-repository-jdbc")  
  
    // Phase 5: Port Wrangler MCP server (expose InfrastructureTools to Cursor / Claude Desktop)  
    implementation("org.springframework.ai:spring-ai-starter-mcp-server")  
  
    // OPTIONAL (Phase 5 stretch): MCP client for MemPalace  
    // implementation("org.springframework.ai:spring-ai-starter-mcp-client")  
  
    // Thin Kafka AdminClient — only for the agent createKafkaTopic @Tool, not a full mgmt module  
    implementation("org.apache.kafka:kafka-clients")  
}
```

Verify the exact chat-memory-repository-jdbc artifact id against start.spring.io for 1.0.3 — the BOM, Ollama, pgvector, and MCP-client coordinates above were confirmed against the GA docs;
the chat-memory starter id follows the same spring-ai-starter-* pattern but should be double-checked when you wire Phase 3.

Appendix C — Sources

- [Spring AI 1.0 GA announcement \(2025-05-20\) \(https://spring.io/blog/2025/05/20/spring-ai-1-0-GA-released/\)](https://spring.io/blog/2025/05/20/spring-ai-1-0-GA-released/)
- [Spring AI 1.0.3 release \(2025-10-01\) \(https://spring.io/blog/2025/10/01/spring-ai-1-0-3-available-now/\)](https://spring.io/blog/2025/10/01/spring-ai-1-0-3-available-now/)
- [Spring AI — Chat Memory reference \(https://docs.spring.io/spring-ai/reference/api/chat-memory.html\)](https://docs.spring.io/spring-ai/reference/api/chat-memory.html)
- [Spring AI — Tool Calling reference \(manual execution, ToolCallingManager, returnDirect, ToolContext\) \(https://docs.spring.io/spring-ai/reference/api/tools.html\)](https://docs.spring.io/spring-ai/reference/api/tools.html)
- [Spring AI — PGvector store reference \(HNSW/IVFFlat, dimensions, distance\) \(https://docs.spring.io/spring-ai/reference/api/vectorlibs/pgvector.html\)](https://docs.spring.io/spring-ai/reference/api/vectorlibs/pgvector.html)
- [Kafka UI tools compared, 2026 \(Kafbat / AKHQ / Redpanda Console\) \(https://factorhouse.io/articles/top-kafka-ui-tools-in-2026-a-practical-comparison-for-engineering-\)](https://factorhouse.io/articles/top-kafka-ui-tools-in-2026-a-practical-comparison-for-engineering-)

[teams\)](#)

- [Docker Model Runner vs Ollama \(2026\) \(https://www.glukhov.org/llm-hosting/comparisons/docker-model-runner-vs-ollama-comparison/\)](https://www.glukhov.org/llm-hosting/comparisons/docker-model-runner-vs-ollama-comparison/)

Grounded against com.dbdeployer @ cafd191. Supersedes the four prior AI roadmaps where they conflict. Generated by Claude (Opus 4.8) via Claude Code.